

CSE2315 Lab Course

Solutions 3

deadline: February 26, 2026, 13:45

1. Suppose we have the following language L over the alphabet $\Sigma = \{a, 0, 1\}$:

$$L = \{ua^k v \mid u, v \in \{0, 1\}^* \text{ and } c_0(u) + c_1(v) > k\}.$$

Here, $c_x(w)$ is defined as the number of occurrences of x in w . Use the pumping lemma to show that L is not regular.

Solution:

Proof. Suppose L is regular. Then there exists a pumping length $p > 0$, in accordance with the pumping lemma. The lemma says all words $w \in L$ with $|w| \geq p$ can be split up into $w = xyz$ with $|xy| \leq p$ and $|y| > 0$ such that $xy^i z \in L$ for all $i \geq 0$.

Now, take $w = 0^p a^p 1$. We have $w \in L$ because $p + 1 > p$. If we split this word according to the above requirements, we get $x = 0^m$, $y = 0^n$ and $z = 0^{p-m-n} a^p 1$ for some $m \geq 0, n > 0$.

Now, take $i = 0$. Then $xy^0 z = 0^m 0^{p-m-n} a^p 1 = 0^{p-n} a^p 1$. Since $n > 0$, $p - n < p$ and so $p - n + 1 \leq p$. Hence $xy^0 z \notin L$. This contradicts the pumping lemma, so L is not regular.

Note: There are also other possibilities for the choices of w and i . □

2. Suppose we have the context-free grammar $G = (\{S, T, V\}, \{a, b, c\}, R, S)$ with R containing the following rules:

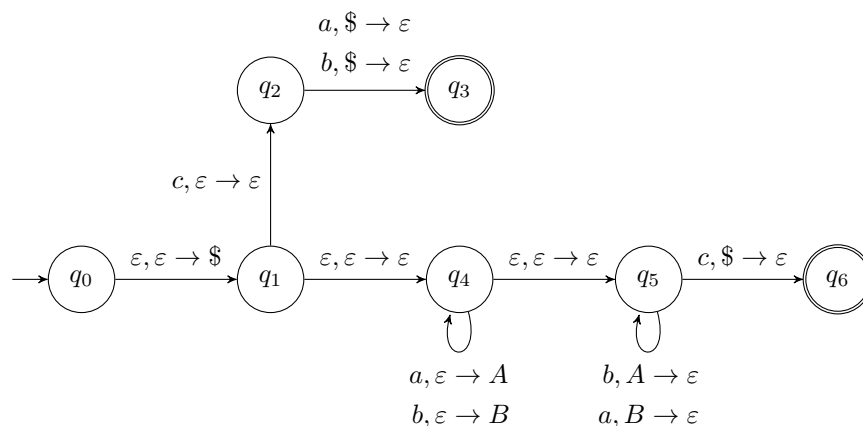
$$\begin{aligned} S &\rightarrow Tc \mid cV \\ T &\rightarrow aTb \mid bTa \mid \varepsilon \\ V &\rightarrow a \mid b \end{aligned}$$

- (a) Describe the set $L(G)$ in your own words.

Solution: The grammar generates the words ca and cb , and the set of words ending in a c for which (after removing this c) the second half is the reverse of the first half, with as and bs swapped.

- (b) Construct a PDA M such that $L(M) = L(G)$. Use no more than 8 states.

Solution:



It is also possible to branch in q_0 instead of in q_1 : the branch corresponding to $S \rightarrow cV$ does not really make use of the $\$$ symbol. Then you could make a direct transition from q_0 to q_4 , using 6 states instead.

(c) Explain (in no more than 12 lines) how your PDA works.

Solution: The PDA first puts a special symbol \$ onto the stack to recognize the bottom. If it reads a c as the first character, it branches to q_2 , which corresponds to the rule $S \rightarrow cV$. In q_2 , it has to read either an a or a b (recall the rule $V \rightarrow a \mid b$). In that case, it pops the \$ symbol again and goes to an accepting state.

In q_1 , the machine can alternatively (nondeterministically) go to q_4 , where it pushes an A (resp. B) for each a (resp. b) it reads. The stack thus contains information on what symbols have been read in what order. Nondeterministically, the PDA goes to q_5 , popping the A s and B s from the stack if it reads the opposite characters in reverse order. (Remember that a stack is LIFO.) If the \$ symbol is on the top of the stack, then all characters have been matched successfully, so it goes to an accepting state.

3. *Old exam question midterm 21-22:* Consider the following rules R of a CFG $G = (V, \Sigma, R, S)$, with $V = \{S, A, B\}$ and $\Sigma = \{a, b\}$:

$$\begin{aligned} S &\rightarrow bB \mid abB \\ A &\rightarrow bA \mid AA \\ B &\rightarrow AS \mid Sa \mid \varepsilon \end{aligned}$$

Convert G into Chomsky Normal Form (CNF). Use the procedure from Sipser, showing intermediate steps.

Solution: First, we add a new start variable:

$$\begin{aligned} S_0 &\rightarrow S \\ S &\rightarrow bB \mid abB \\ A &\rightarrow bA \mid AA \\ B &\rightarrow AS \mid Sa \mid \varepsilon \end{aligned}$$

Second, we take care of the ϵ -rules:

$$\begin{aligned} S_0 &\rightarrow S \\ S &\rightarrow bB \mid abB \mid b \mid ab \\ A &\rightarrow bA \mid AA \\ B &\rightarrow AS \mid Sa \end{aligned}$$

Third, we handle all unit rules:

$$\begin{aligned} S_0 &\rightarrow bB \mid abB \mid b \mid ab \\ S &\rightarrow bB \mid abB \mid b \mid ab \\ A &\rightarrow bA \mid AA \\ B &\rightarrow AS \mid Sa \end{aligned}$$

Finally, we convert all remaining rules in proper form:

$$\begin{aligned} S_0 &\rightarrow U_b B \mid U_a S_1 \mid b \mid U_a U_b \\ S &\rightarrow U_b B \mid U_a S_1 \mid b \mid U_a U_b \\ S_1 &\rightarrow U_b B \\ A &\rightarrow U_b A \mid AA \\ B &\rightarrow AS \mid S U_a \\ U_a &\rightarrow a \\ U_b &\rightarrow b \end{aligned}$$

4. You are designing a new data format called TUDON (short for TU Delft Object Notation¹). In TUDON, you can represent dictionaries of key/value pairs. Such a data format is useful when sending data between computers: the receiver can lookup the value the sender supplied for a key.

More precisely, each key in a dictionary is a sequence of lowercase letters (at least one letter). The value in a dictionary is either also such a sequence, or a dictionary itself. A dictionary starts with a < and ends with a >. Each key/value pair is separated by a colon, and key/value pairs themselves are separated by a semicolon. Each word in TUDON is a dictionary.

Examples of TUDON words are:

<>

<hello:world>

<a:b;test:<example:x;abc:<def:ghi>>;xyz:aaa>

Let L be the set of all TUDON words.

- (a) Give a CFG G such that $L(G) = L$, using no more than 6 variables and $9 + 26$ rules. (You'll need 26 rules to represent all lowercase letters. You are allowed to omit (abbreviate) most of these rules as long as it's clear what you mean.)

Solution: The grammar $G = (\{S, T, E, V, N, L\}, \{<, >, :, ;, a \mid \dots \mid z\}, R, S)$ where R consists of the following rules, generates L :

$$S \rightarrow <T> \mid <>$$

$$T \rightarrow E \mid E;T$$

$$E \rightarrow N:V$$

$$V \rightarrow S \mid N$$

$$N \rightarrow LN \mid L$$

$$L \rightarrow a \mid \dots \mid z$$

- (b) Explain why your grammar generates L . (For example, explain the purpose/meaning of each rule.)

Solution: Each rule corresponds to a property of TUDON:

The first rule defines a dictionary.

The second rule defines a list of key/value pairs.

The third rule defines a key/value pair.

The fourth rule defines the value of a key/value pair: there are two possibilities.

The fifth and sixth rules define a sequence of lowercase letters of length at least one.

- (c) Is your grammar ambiguous? If so, give a word w that shows ambiguity. If not, make plausible why.

Solution: The above grammar is not ambiguous.

A sequence of lowercase letters has only one leftmost derivation starting from N : letters can only be *prepended*, through the rule $N \rightarrow LN$.

A dictionary also has only one leftmost derivation starting from S : there is (quite similarly) only one way to extend a list of key/value pairs. (This would not be the case if there was an additional rule $T \rightarrow T;E$.)

5. *Revision, old exam question resit 19-20:* We're introducing a new operator @ that transforms all words of a language in words of the same length but only consist of a's, formally defined as $A@ = \{a^{|w|} \mid w \in A\}$. Prove that regularity is closed under @, meaning if A is regular, then $A@$ is also regular.

¹See also <http://json.org>, which contains a CFG you might want to use as inspiration.

Solution: Say A is a regular language. Then, there exists a NFA N that recognizes A . Now consider NFA N' which is N except $\Sigma' = \{a\}$ and all symbols previously in Σ are replaced with a . If a word w of length n is in A , N accepts it. Now, x^n is in $A@$, because N' accepts it via the same transitions as N accepted w . If a^n is in $A@$, then N' accepts it. Now, using the same transitions as the ones N' used to accept a^n , there must exist a word of length n that uses these transitions to reach that same accepting state. This proves that N' recognizes $A@$, which means $A@$ is also regular.

6. Revision, old exam question resit 22-23:

- (a) Explain why the following claim is true: Let N be an arbitrary NFA with k states. If $L(N) \neq \emptyset$, then there is a word of length $\leq k$ in $L(N)$.

Solution: Let q_f be a random reachable state in F of N . Since $L(N) \neq \emptyset$ such a state must exist. Any cycles in the path from q_0 through q_f can be removed by just shortening the word. As such we may assume that if there is a path from q_0 to q_f there is also a simple path. A simple path visits at most all states once and thus reads no more than k letters.

Taken from: Sipser 1.58a

- (b) Provide a counterexample and explanation showing the following claim is false: Let D be an arbitrary DFA D with $k > 1$ states. There is a DFA D' with $k - 1$ states, such that $L(D') = L(D)$. Your counterexample only needs to give a formal representation or a transition diagram of D , you should then explain why no D' can exist.

Solution: Take the DFA studied in class that accepts iff the number of a s is even. This has two states alternating which state we are in on every a . (We assume for simplicity that $\Sigma = \{a\}$).

With one state all we can accept is Σ^* or $\emptyset$, as this one state is either accepting or rejecting respectively. Clearly this is neither.

Taken from: Sipser 1.44

See next page for an optional extra exercise puzzle.

Optional extra exercise This last exercise is not part of the learning material. However, if you like puzzles and have some time left, you might like to do this challenging puzzle.

After the adventure of last week, there is still room for improvement. You realize that last week, the robot might have immediately vanished if you had chosen the right teleportation device at the start instead of the left one. Therefore, you have decided to make multiple copies of the robot, and to only pick up the treasure once you have explored the mansion sufficiently. For the first copy, you press a few buttons to explore (given below).

Start	Left	Left	Right	Left	Right	Right	Right	Left
{Red}	{Yellow}	{Yellow}	{Green}	{Red}	{Yellow}	{Green}	{Blue}	{}

This is even more trivial than you thought. Apparently, there are only 4 room colours, and since there are still a maximum of 4 rooms, each room has a unique colour. You can easily map out which teleportation devices in which rooms lead to which rooms. Clearly, the teleporation device that leads outside is the right one in the blue room. You can get there with $left \rightarrow right \rightarrow right \rightarrow right$. So you send out the second copy of the robot, which appears in the red room, and let it pick up the treasure. You press left, letting the robot arrive in the yellow room. However, when you then press right, you get a message telling you the robot is in a yellow room. That's weird. Earlier, the right teleportation device in the yellow room lead to the green room.

You decide to study a little more, and then you find it. Apparently, in your enthousiasm, you didn't read everything about this mansion. The teleportation devices in this mansion sometimes attach components to the robot. If the robot has no components, the teleportation device deconstructs and reconstructs it in 0 or more rooms as usual, but it might also attach 1 component to it (this can be different for each room). If the robot does have a component, the most recently placed component determines in which 0 or more **additional** rooms the robot is constructed. In these additional constructions (so not in the other constructions), the most recently placed component is removed, but another 1 component might be attached (again, this can be a different component per construction). Finally, there are 2 different components.

Knowing this, you think you know what to do. Maybe you need to press the left button and then press the right button. But again, you seem stuck in the yellow room. What is the quickest way that makes sure to get the treasure out?

Start	Left	Right	Left	Right
{Red}	{Yellow}	{Yellow}	{Yellow}	{Yellow}

- Each room has 2 teleportation devices, a left one and a right one
- Teleportation devices are deterministic
 - The same device in the same room will always lead to a given set of rooms, **each with 0 or 1 given extra component**
 - The same device in the same room **with the same most recent component** will always lead to a given set of **additional rooms, each with the most recent component removed and 0 or 1 given extra component**
- If 2 (or more) teleportation devices teleport a robot **with the same components** to the same room at the same time, only 1 of those will be build (the device cannot see that they are 2 signals)
- At least 1 teleportation device leads outside, **irrespective of its components**
- There are at most 4 rooms
- Rooms will not change colour (the same room will always have the same colour)
- **There are 2 different components**
- After the robot has taken the teleportation devices as given in the table (and entered rooms with wall colours as given in the table), what is the quickest way to get the robot outside to collect the treasure?

Solution: $Right \rightarrow Left \rightarrow Right \rightarrow Right \rightarrow Right \rightarrow Right$